

14.3 Graph Traversals

Greek mythology tells of an elaborate labyrinth that was built to house the monstrous Minotaur, which was part bull and part man. This labyrinth was so complex that neither beast nor human could escape it. No human, that is, until the Greek hero, Theseus, with the help of the king's daughter, Ariadne, decided to implement a **graph traversal** algorithm. Theseus fastened a ball of thread to the door of the labyrinth and unwound it as he traversed the twisting passages in search of the monster. Theseus obviously knew about good algorithm design, for, after finding and defeating the beast, Theseus easily followed the string back out of the labyrinth to the loving arms of Ariadne.

Formally, a **traversal** is a systematic procedure for exploring a graph by examining all of its vertices and edges. A traversal is efficient if it visits all the vertices and edges in time proportional to their number, that is, in linear time.

Graph traversal algorithms are key to answering many fundamental questions about graphs involving the notion of **reachability**, that is, in determining how to travel from one vertex to another while following paths of a graph. Interesting problems that deal with reachability in an undirected graph G include the following:

- Computing a path from vertex u to vertex v , or reporting that no such path exists.
- Given a start vertex s of G , computing, for every vertex v of G , a path with the minimum number of edges between s and v , or reporting that no such path exists.
- Testing whether G is connected.
- Computing a spanning tree of G , if G is connected.
- Computing the connected components of G .
- Identifying a cycle in G , or reporting that G has no cycles.

Interesting problems that deal with reachability in a directed graph $\vec{G}$ include the following:

- Computing a directed path from vertex u to vertex v , or reporting that no such path exists.
- Finding all the vertices of $\vec{G}$ that are reachable from a given vertex s .
- Determine whether $\vec{G}$ is acyclic.
- Determine whether $\vec{G}$ is strongly connected.

In the remainder of this section, we will present two efficient graph traversal algorithms, called **depth-first search** and **breadth-first search**, respectively.

14.3.1 Depth-First Search

The first traversal algorithm we consider in this section is *depth-first search* (DFS). Depth-first search is useful for testing a number of properties of graphs, including whether there is a path from one vertex to another and whether or not a graph is connected.

Depth-first search in a graph G is analogous to wandering in a labyrinth with a string and a can of paint without getting lost. We begin at a specific starting vertex s in G , which we initialize by fixing one end of our string to s and painting s as “visited.” The vertex s is now our “current” vertex. In general, if we call our current vertex u , we traverse G by considering an arbitrary edge (u, v) incident to the current vertex u . If the edge (u, v) leads us to a vertex v that is already visited (that is, painted), we ignore that edge. If, on the other hand, (u, v) leads to an unvisited vertex v , then we unroll our string, and go to v . We then paint v as “visited,” and make it the current vertex, repeating the computation above. Eventually, we will get to a “dead end,” that is, a current vertex v such that all the edges incident to v lead to vertices already visited. To get out of this impasse, we roll our string back up, backtracking along the edge that brought us to v , going back to a previously visited vertex u . We then make u our current vertex and repeat the computation above for any edges incident to u that we have not yet considered. If all of u ’s incident edges lead to visited vertices, then we again roll up our string and backtrack to the vertex we came from to get to u , and repeat the procedure at that vertex. Thus, we continue to backtrack along the path that we have traced so far until we find a vertex that has yet unexplored edges, take one such edge, and continue the traversal. The process terminates when our backtracking leads us back to the start vertex s , and there are no more unexplored edges incident to s .

The pseudocode for a depth-first search traversal starting at a vertex u (see Code Fragment 14.4) follows our analogy with string and paint. We use recursion to implement the string analogy, and we assume that we have a mechanism (the paint analogy) to determine whether a vertex or edge has been previously explored.

Algorithm DFS(G, u):

Input: A graph G and a vertex u of G

Output: A collection of vertices reachable from u , with their discovery edges

Mark vertex u as visited.

for each of u ’s outgoing edges, $e = (u, v)$ **do**

if vertex v has not been visited **then**

 Record edge e as the discovery edge for vertex v .

 Recursively call DFS(G, v).

Code Fragment 14.4: The DFS algorithm.

Classifying Graph Edges with DFS

An execution of depth-first search can be used to analyze the structure of a graph, based upon the way in which edges are explored during the traversal. The DFS process naturally identifies what is known as the **depth-first search tree** rooted at a starting vertex s . Whenever an edge $e = (u, v)$ is used to discover a new vertex v during the DFS algorithm of Code Fragment 14.4, that edge is known as a **discovery edge** or **tree edge**, as oriented from u to v . All other edges that are considered during the execution of DFS are known as **nontree edges**, which take us to a previously visited vertex. In the case of an undirected graph, we will find that all nontree edges that are explored connect the current vertex to one that is an ancestor of it in the DFS tree. We will call such an edge a **back edge**. When performing a DFS on a directed graph, there are three possible kinds of nontree edges:

- **back edges**, which connect a vertex to an ancestor in the DFS tree
- **forward edges**, which connect a vertex to a descendant in the DFS tree
- **cross edges**, which connect a vertex to a vertex that is neither its ancestor nor its descendant

An example application of the DFS algorithm on a directed graph is shown in Figure 14.8, demonstrating each type of nontree edge. An example application of the DFS algorithm on an undirected graph is shown in Figure 14.9.

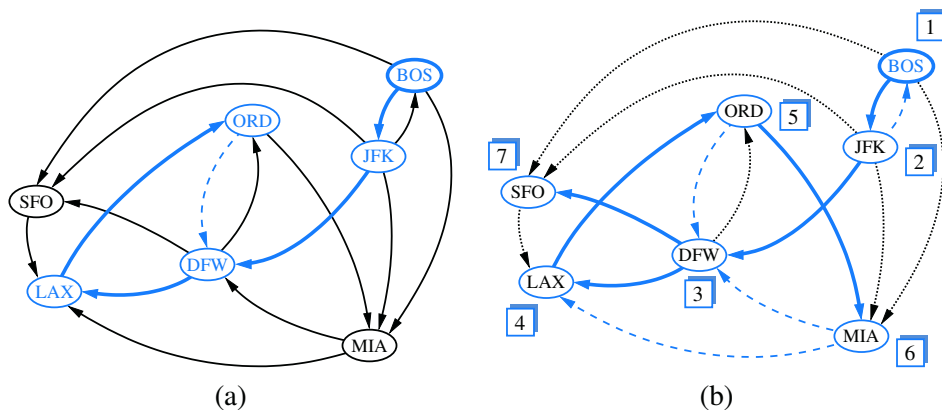


Figure 14.8: An example of a DFS in a directed graph, starting at vertex (BOS): (a) intermediate step, where, for the first time, a considered edge leads to an already visited vertex (DFW); (b) the completed DFS. The tree edges are shown with thick blue lines, the back edges are shown with dashed blue lines, and the forward and cross edges are shown with dotted black lines. The order in which the vertices are visited is indicated by a label next to each vertex. The edge (ORD,DFW) is a back edge, but (DFW,ORD) is a forward edge. Edge (BOS,SFO) is a forward edge, and (SFO,LAX) is a cross edge.

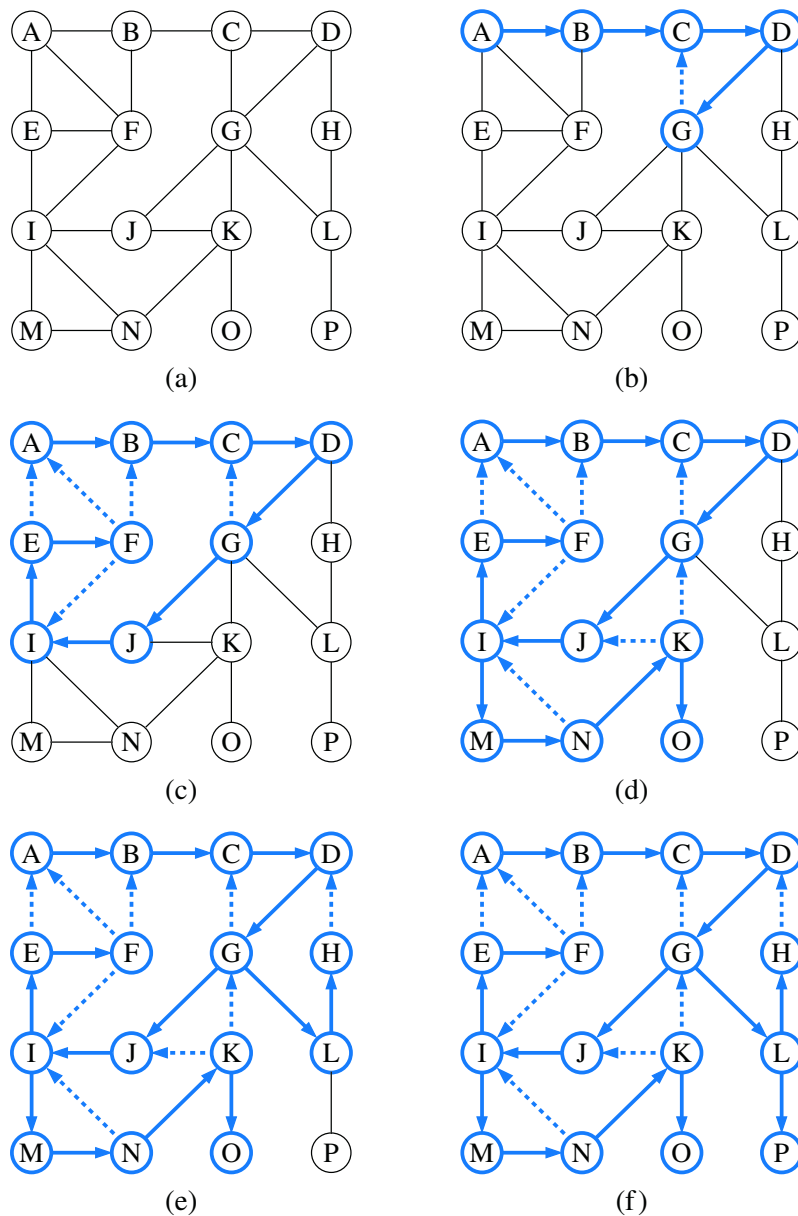


Figure 14.9: Example of depth-first search traversal on an undirected graph starting at vertex A. We assume that a vertex's adjacencies are considered in alphabetical order. Visited vertices and explored edges are highlighted, with discovery edges drawn as solid lines and nontree (back) edges as dashed lines: (a) input graph; (b) path of tree edges, traced from A until back edge (G,C) is examined; (c) reaching F, which is a dead end; (d) after backtracking to I, resuming with edge (I,M), and hitting another dead end at O; (e) after backtracking to G, continuing with edge (G,L), and hitting another dead end at H; (f) final result.

Properties of a Depth-First Search

There are a number of observations that we can make about the depth-first search algorithm, many of which derive from the way the DFS algorithm partitions the edges of a graph G into groups. We will begin with the most significant property.

Proposition 14.12: *Let G be an undirected graph on which a DFS traversal starting at a vertex s has been performed. Then the traversal visits all vertices in the connected component of s , and the discovery edges form a spanning tree of the connected component of s .*

Justification: Suppose there is at least one vertex w in s 's connected component not visited, and let v be the first unvisited vertex on some path from s to w (we may have $v = w$). Since v is the first unvisited vertex on this path, it has a neighbor u that was visited. But when we visited u , we must have considered the edge (u, v) ; hence, it cannot be correct that v is unvisited. Therefore, there are no unvisited vertices in s 's connected component.

Since we only follow a discovery edge when we go to an unvisited vertex, we will never form a cycle with such edges. Therefore, the discovery edges form a connected subgraph without cycles, hence a tree. Moreover, this is a spanning tree because, as we have just seen, the depth-first search visits each vertex in the connected component of s . ■

Proposition 14.13: *Let $\vec{G}$ be a directed graph. Depth-first search on $\vec{G}$ starting at a vertex s visits all the vertices of $\vec{G}$ that are reachable from s . Also, the DFS tree contains directed paths from s to every vertex reachable from s .*

Justification: Let V_s be the subset of vertices of $\vec{G}$ visited by DFS starting at vertex s . We want to show that V_s contains s and every vertex reachable from s belongs to V_s . Suppose now, for the sake of a contradiction, that there is a vertex w reachable from s that is not in V_s . Consider a directed path from s to w , and let (u, v) be the first edge on such a path taking us out of V_s , that is, u is in V_s but v is not in V_s . When DFS reaches u , it explores all the outgoing edges of u , and thus must also reach vertex v via edge (u, v) . Hence, v should be in V_s , and we have obtained a contradiction. Therefore, V_s must contain every vertex reachable from s .

We prove the second fact by induction on the steps of the algorithm. We claim that each time a discovery edge (u, v) is identified, there exists a directed path from s to v in the DFS tree. Since u must have previously been discovered, there exists a path from s to u , so by appending the edge (u, v) to that path, we have a directed path from s to v . ■

Note that since back edges always connect a vertex v to a previously visited vertex u , each back edge implies a cycle in G , consisting of the discovery edges from u to v plus the back edge (u, v) .

Running Time of Depth-First Search

In terms of its running time, depth-first search is an efficient method for traversing a graph. Note that DFS is called at most once on each vertex (since it gets marked as visited), and therefore every edge is examined at most twice for an undirected graph, once from each of its end vertices, and at most once in a directed graph, from its origin vertex. If we let $n_s \leq n$ be the number of vertices reachable from a vertex s , and $m_s \leq m$ be the number of incident edges to those vertices, a DFS starting at s runs in $O(n_s + m_s)$ time, provided the following conditions are satisfied:

- The graph is represented by a data structure such that creating and iterating through the `outgoingEdges( $v$ )` takes $O(\deg(v))$ time, and the `opposite( $v, e$ )` method takes $O(1)$ time. The adjacency list structure is one such structure, but the adjacency matrix structure is not.
- We have a way to “mark” a vertex or edge as explored, and to test if a vertex or edge has been explored in $O(1)$ time. We discuss ways of implementing DFS to achieve this goal in the next section.

Given the assumptions above, we can solve a number of interesting problems.

Proposition 14.14: *Let G be an undirected graph with n vertices and m edges. A DFS traversal of G can be performed in $O(n + m)$ time, and can be used to solve the following problems in $O(n + m)$ time:*

- Computing a path between two given vertices of G , if one exists.
- Testing whether G is connected.
- Computing a spanning tree of G , if G is connected.
- Computing the connected components of G .
- Computing a cycle in G , or reporting that G has no cycles.

Proposition 14.15: *Let $\vec{G}$ be a directed graph with n vertices and m edges. A DFS traversal of $\vec{G}$ can be performed in $O(n + m)$ time, and can be used to solve the following problems in $O(n + m)$ time:*

- Computing a directed path between two given vertices of $\vec{G}$, if one exists.
- Computing the set of vertices of $\vec{G}$ that are reachable from a given vertex s .
- Testing whether $\vec{G}$ is strongly connected.
- Computing a directed cycle in $\vec{G}$, or reporting that $\vec{G}$ is acyclic.

The justification of Propositions 14.14 and 14.15 is based on algorithms that use slightly modified versions of the DFS algorithm as subroutines. We will explore some of those extensions in the remainder of this section.

14.3.2 DFS Implementation and Extensions

We will begin by providing a Java implementation of the depth-first search algorithm. We originally described the algorithm with pseudocode in Code Fragment 14.4. In order to implement it, we must have a mechanism for keeping track of which vertices have been visited, and for recording the resulting DFS tree edges. For this bookkeeping, we use two auxiliary data structures. First, we maintain a set, named *known*, containing vertices that have already been visited. Second, we keep a map, named *forest*, that associates, with a vertex v , the edge e of the graph that is used to discover v (if any). Our DFS method is presented in Code Fragment 14.5.

```

1  /** Performs depth-first search of Graph g starting at Vertex u. */
2  public static <V,E> void DFS(Graph<V,E> g, Vertex<V> u,
3      Set<Vertex<V>> known, Map<Vertex<V>,Edge<E>> forest) {
4      known.add(u); // u has been discovered
5      for (Edge<E> e : g.outgoingEdges(u)) { // for every outgoing edge from u
6          Vertex<V> v = g.opposite(u, e);
7          if (!known.contains(v)) {
8              forest.put(v, e); // e is the tree edge that discovered v
9              DFS(g, v, known, forest); // recursively explore from v
10         }
11     }
12 }
```

Code Fragment 14.5: Recursive implementation of depth-first search on a graph, starting at a designated vertex u . As an outcome of a call, visited vertices are added to the *known* set, and discovery edges are added to the *forest*.

Our DFS method does not make any assumption about how the *Set* or *Map* instances are implemented; however, the $O(n + m)$ running-time analysis of the previous section does presume that we can “mark” a vertex as explored or test the status of a vertex in $O(1)$ time. If we use hash-based implementations of the set and map structure, then all of their operations run in $O(1)$ *expected* time, and the overall algorithm runs in $O(n + m)$ time with very high probability. In practice, this is a compromise we are willing to accept.

If vertices can be numbered from $0, \dots, n - 1$ (a common assumption for graph algorithms), then the set and map can be implemented more directly as a lookup table, with a vertex label used as an index into an array of size n . In that case, the necessary set and map operations run in worst-case $O(1)$ time. Alternatively, we can “decorate” each vertex with the auxiliary information, either by leveraging the generic type of the element that is stored with each vertex, or by redesigning the *Vertex* type to store additional fields. That would allow marking operations to be performed in $O(1)$ -time, without any assumption about vertices being numbered.

Reconstructing a Path from u to v

We can use the basic DFS method as a tool to identify the (directed) path leading from vertex u to v , if v is reachable from u . This path can easily be reconstructed from the information that was recorded in the forest of discovery edges during the traversal. Code Fragment 14.6 provides an implementation of a secondary method that produces an ordered list of vertices on the path from u to v , if given the map of discovery edges that was computed by the original DFS method.

To reconstruct the path, we begin at the *end* of the path, examining the forest of discovery edges to determine what edge was used to reach vertex v . We then determine the opposite vertex of that edge and repeat the process to determine what edge was used to discover it. By continuing this process until reaching u , we can construct the entire path. Assuming constant-time lookup in the forest map, the path reconstruction takes time proportional to the length of the path, and therefore, it runs in $O(n)$ time (in addition to the time originally spent calling DFS).

```

1  /** Returns an ordered list of edges comprising the directed path from u to v. */
2  public static <V,E> PositionalList<Edge<E>>
3  constructPath(Graph<V,E> g, Vertex<V> u, Vertex<V> v,
4               Map<Vertex<V>,Edge<E>> forest) {
5      PositionalList<Edge<E>> path = new LinkedPositionalList<>();
6      if (forest.get(v) != null) {           // v was discovered during the search
7          Vertex<V> walk = v;               // we construct the path from back to front
8          while (walk != u) {
9              Edge<E> edge = forest.get(walk);
10             path.addFirst(edge);           // add edge to *front* of path
11             walk = g.opposite(walk, edge); // repeat with opposite endpoint
12         }
13     }
14     return path;
15 }
```

Code Fragment 14.6: Method to reconstruct a directed path from u to v , given the trace of discovery from a DFS started at u . The method returns an ordered list of vertices on the path.

Testing for Connectivity

We can use the basic DFS method to determine whether a graph is connected. In the case of an undirected graph, we simply start a depth-first search at an arbitrary vertex and then test whether `known.size()` equals n at the conclusion. If the graph is connected, then by Proposition 14.12, all vertices will have been discovered; conversely, if the graph is not connected, there must be at least one vertex v that is not reachable from u , and that will not be discovered.

For directed graph, $\vec{G}$, we may wish to test whether it is **strongly connected**, that is, whether for every pair of vertices u and v , both u reaches v and v reaches u . If we start an independent call to DFS from each vertex, we could determine whether this was the case, but those n calls when combined would run in $O(n(n+m))$. However, we can determine if $\vec{G}$ is strongly connected much faster than this, requiring only two depth-first searches.

We begin by performing a depth-first search of our directed graph $\vec{G}$ starting at an arbitrary vertex s . If there is any vertex of $\vec{G}$ that is not visited by this traversal, and is not reachable from s , then the graph is not strongly connected. If this first depth-first search visits each vertex of $\vec{G}$, we need to then check whether s is reachable from all other vertices. Conceptually, we can accomplish this by making a copy of graph $\vec{G}$, but with the orientation of all edges reversed. A depth-first search starting at s in the reversed graph will reach every vertex that could reach s in the original. In practice, a better approach than making a new graph is to reimplement a version of the DFS method that loops through all **incoming** edges to the current vertex, rather than all **outgoing** edges. Since this algorithm makes just two DFS traversals of $\vec{G}$, it runs in $O(n+m)$ time.

Computing All Connected Components

When a graph is not connected, the next goal we may have is to identify all of the **connected components** of an undirected graph, or the **strongly connected components** of a directed graph. We will begin by discussing the undirected case.

If an initial call to DFS fails to reach all vertices of a graph, we can restart a new call to DFS at one of those unvisited vertices. An implementation of such a comprehensive DFSComplete method is given in Code Fragment 14.7. It returns a map that represents a **DFS forest** for the entire graph. We say this is a forest rather than a tree, because the graph may not be connected.

Vertices that serve as roots of DFS trees within this forest will not have discovery edges and will not appear as keys in the returned map. Therefore, the number of connected components of the graph g is equal to $g.\text{numVertices}() - \text{forest.size}()$.

```

1  /** Performs DFS for the entire graph and returns the DFS forest as a map. */
2  public static <V,E> Map<Vertex<V>,Edge<E>> DFSComplete(Graph<V,E> g) {
3      Set<Vertex<V>> known = new HashSet<>();
4      Map<Vertex<V>,Edge<E>> forest = new ProbeHashMap<>();
5      for (Vertex<V> u : g.vertices())
6          if (!known.contains(u))
7              DFS(g, u, known, forest);           // (re)start the DFS process at u
8      return forest;
9  }
```

Code Fragment 14.7: Top-level method that returns a DFS forest for an entire graph.

We can further determine which vertices are in which component, either by examining the structure of the forest that is returned, or by making a minor modification to the core DFS method to tag each vertex with a component number when it is first discovered. (See Exercise C-14.43.)

Although the `DFSComplete` method makes multiple calls to the original DFS method, the total time spent by a call to `DFSComplete` is $O(n + m)$. For an undirected graph, recall from our original analysis on page 635 that a single call to DFS starting at vertex s runs in time $O(n_s + m_s)$ where n_s is the number of vertices reachable from s , and m_s is the number of incident edges to those vertices. Because each call to DFS explores a different component, the sum of $n_s + m_s$ terms is $n + m$.

The situation is more complex for finding strongly connected components of a directed graph. The $O(n + m)$ total bound for a call to `DFSComplete` applies to the directed case as well, because when restarting the process, we proceed with the existing set of known vertices. This ensures that the DFS subroutine is called once on each vertex, and therefore that each outgoing edge is explored only once during the entire process.

As an example, consider again the graph of Figure 14.8. If we were to start the original DFS method at vertex ORD, the known set of vertices would become { ORD, DFW, SFO, LAX, MIA }. If restarting the DFS method at vertex BOS, the outgoing edges to vertices SFO and MIA would not result in further recursion, because those vertices are marked as known.

However, the forest returned by a single call to `DFSComplete` does not represent the strongly connected components of the graph. There exists an approach for computing those components in $O(n + m)$ time, making use of two calls to `DFSComplete`, but the details are beyond the scope of this book.

Detecting Cycles with DFS

For both undirected and directed graphs, a cycle exists if and only if a **back edge** exists relative to the DFS traversal of that graph. It is easy to see that if a back edge exists, a cycle exists by taking the back edge from the descendant to its ancestor and then following the tree edges back to the descendant. Conversely, if a cycle exists in the graph, there must be a back edge relative to a DFS (although we do not prove this fact here).

Algorithmically, detecting a back edge in the undirected case is easy, because all edges are either tree edges or back edges. In the case of a directed graph, additional modifications to the core DFS implementation are needed to properly categorize a nontree edge as a back edge. When a directed edge is explored leading to a previously visited vertex, we must recognize whether that vertex is an ancestor of the current vertex. This can be accomplished, for example, by maintaining another set, with all vertices upon which a recursive call to DFS is currently active. We leave details as an exercise (C-14.42).

14.3.3 Breadth-First Search

The advancing and backtracking of a depth-first search, as described in the previous section, defines a traversal that could be physically traced by a single person exploring a graph. In this section, we will consider another algorithm for traversing a connected component of a graph, known as a *breadth-first search* (BFS). The BFS algorithm is more akin to sending out, in all directions, many explorers who collectively traverse a graph in coordinated fashion.

A BFS proceeds in rounds and subdivides the vertices into *levels*. BFS starts at vertex s , which is at level 0. In the first round, we paint as “visited,” all vertices adjacent to the start vertex s ; these vertices are one step away from the beginning and are placed into level 1. In the second round, we allow all explorers to go two steps (i.e., edges) away from the starting vertex. These new vertices, which are adjacent to level 1 vertices and not previously assigned to a level, are placed into level 2 and marked as “visited.” This process continues in similar fashion, terminating when no new vertices are found in a level.

A Java implementation of BFS is given in Code Fragment 14.8. We follow a convention similar to that of DFS (Code Fragment 14.5), maintaining a known set of vertices, and storing the BFS tree edges in a map. We illustrate a BFS traversal in Figure 14.10.

```

1  /** Performs breadth-first search of Graph g starting at Vertex u. */
2  public static <V,E> void BFS(Graph<V,E> g, Vertex<V> s,
3      Set<Vertex<V>> known, Map<Vertex<V>,Edge<E>> forest) {
4      PositionalList<Vertex<V>> level = new LinkedPositionalList<>();
5      known.add(s);
6      level.addLast(s);                                // first level includes only s
7      while (!level.isEmpty()) {
8          PositionalList<Vertex<V>> nextLevel = new LinkedPositionalList<>();
9          for (Vertex<V> u : level)
10             for (Edge<E> e : g.outgoingEdges(u)) {
11                 Vertex<V> v = g.opposite(u, e);
12                 if (!known.contains(v)) {
13                     known.add(v);
14                     forest.put(v, e);                // e is the tree edge that discovered v
15                     nextLevel.addLast(v);            // v will be further considered in next pass
16                 }
17             }
18             level = nextLevel;                        // relabel 'next' level to become the current
19         }
20     }

```

Code Fragment 14.8: Implementation of breadth-first search on a graph, starting at a designated vertex s .

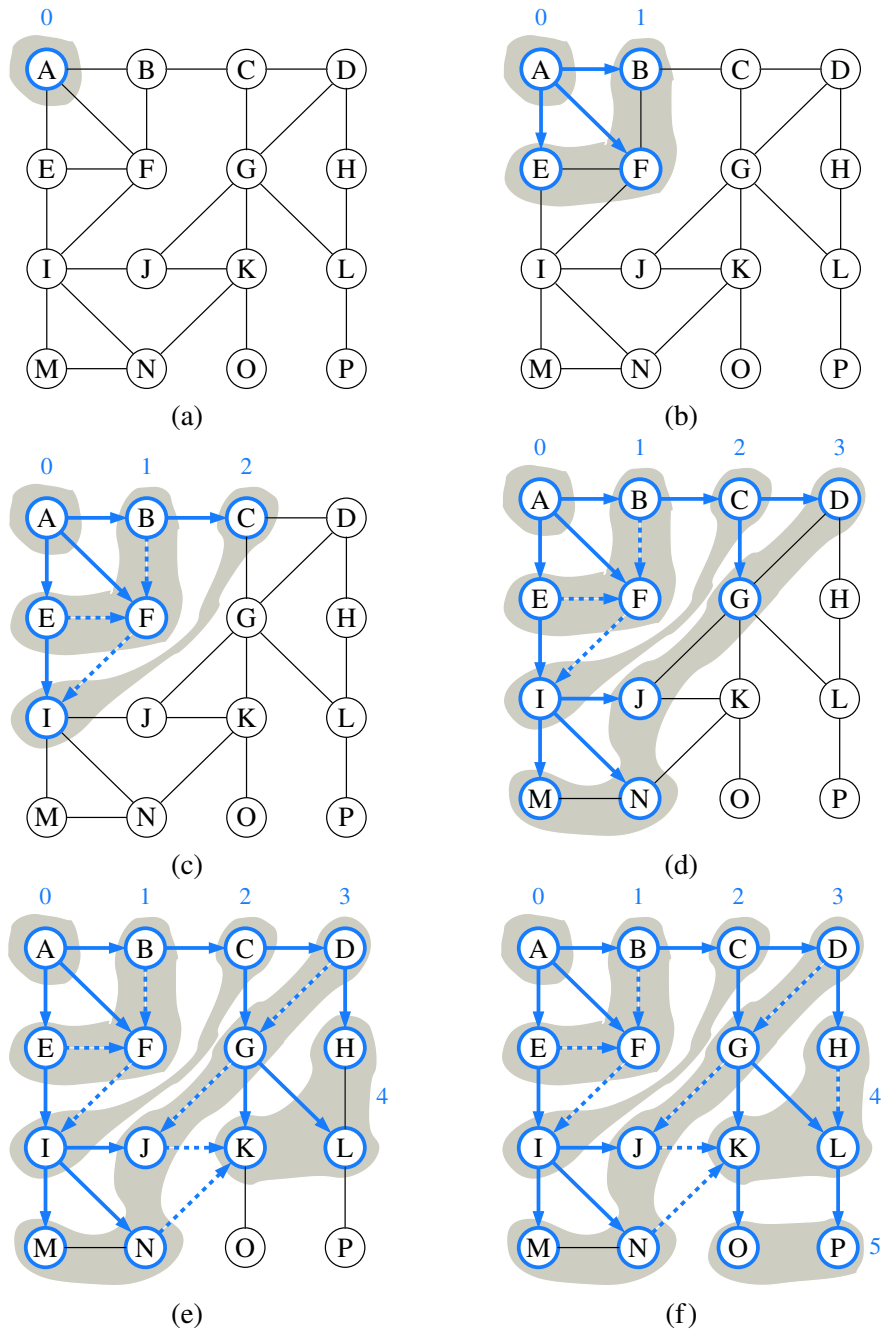


Figure 14.10: Example of breadth-first search traversal, where the edges incident to a vertex are considered in alphabetical order of the adjacent vertices. The discovery edges are shown with solid lines and the nontree (cross) edges are shown with dashed lines: (a) starting the search at A; (b) discovery of level 1; (c) discovery of level 2; (d) discovery of level 3; (e) discovery of level 4; (f) discovery of level 5.

When discussing DFS, we described a classification of nontree edges being either *back edges*, which connect a vertex to one of its ancestors, *forward edges*, which connect a vertex to one of its descendants, or *cross edges*, which connect a vertex to another vertex that is neither its ancestor nor its descendant. For BFS on an undirected graph, all nontree edges are cross edges (see Exercise C-14.46), and for BFS on a directed graph, all nontree edges are either back edges or cross edges (see Exercise C-14.47).

The BFS traversal algorithm has a number of interesting properties, some of which we explore in the proposition that follows. Most notably, a path in a breadth-first search tree rooted at vertex s to any other vertex v is guaranteed to be the shortest such path from s to v in terms of the number of edges.

Proposition 14.16: *Let G be an undirected or directed graph on which a BFS traversal starting at vertex s has been performed. Then*

- *The traversal visits all vertices of G that are reachable from s .*
- *For each vertex v at level i , the path of the BFS tree T between s and v has i edges, and any other path of G from s to v has at least i edges.*
- *If (u, v) is an edge that is not in the BFS tree, then the level number of v can be at most 1 greater than the level number of u .*

We leave the justification of this proposition as Exercise C-14.49.

The analysis of the running time of BFS is similar to the one of DFS, with the algorithm running in $O(n + m)$ time, or more specifically, in $O(n_s + m_s)$ time if n_s is the number of vertices reachable from vertex s , and $m_s \leq m$ is the number of incident edges to those vertices. To explore the entire graph, the process can be restarted at another vertex, akin to the DFSCoordinate method of Code Fragment 14.7. The actual path from vertex s to vertex v can be reconstructed using the `constructPath` method of Code Fragment 14.6

Proposition 14.17: *Let G be a graph with n vertices and m edges represented with the adjacency list structure. A BFS traversal of G takes $O(n + m)$ time.*

Although our implementation of BFS in Code Fragment 14.8 progresses level by level, the BFS algorithm can also be implemented using a single FIFO queue to represent the current fringe of the search. Starting with the source vertex in the queue, we repeatedly remove the vertex from the front of the queue and insert any of its unvisited neighbors to the back of the queue. (See Exercise C-14.50.)

In comparing the capabilities of DFS and BFS, both can be used to efficiently find the set of vertices that are reachable from a given source, and to determine paths to those vertices. However, BFS guarantees that those paths use as few edges as possible. For an undirected graph, both algorithms can be used to test connectivity, to identify connected components, or to locate a cycle. For directed graphs, the DFS algorithm is better suited for certain tasks, such as finding a directed cycle in the graph, or in identifying the strongly connected components.